Regression

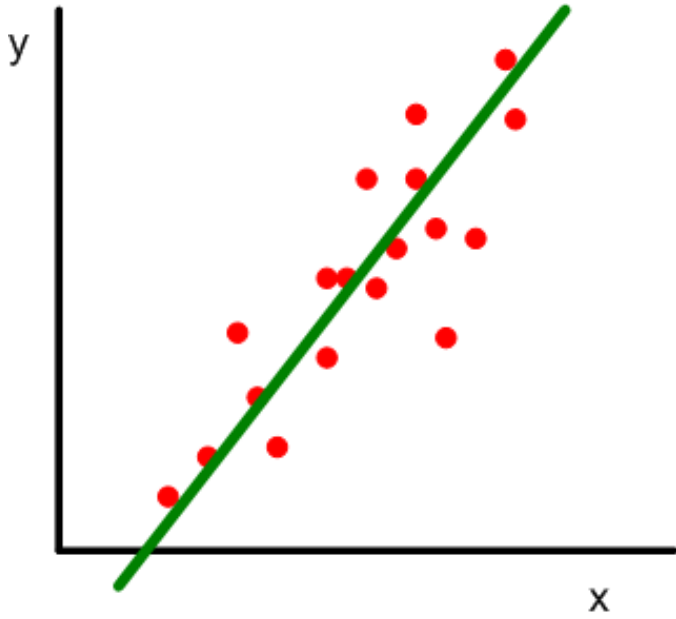
Machine learning

Ing. Tomáš Vičar, Ph.D.
Department of Biomedical Engineering,
FEEC, Brno University of Technology

Introduction

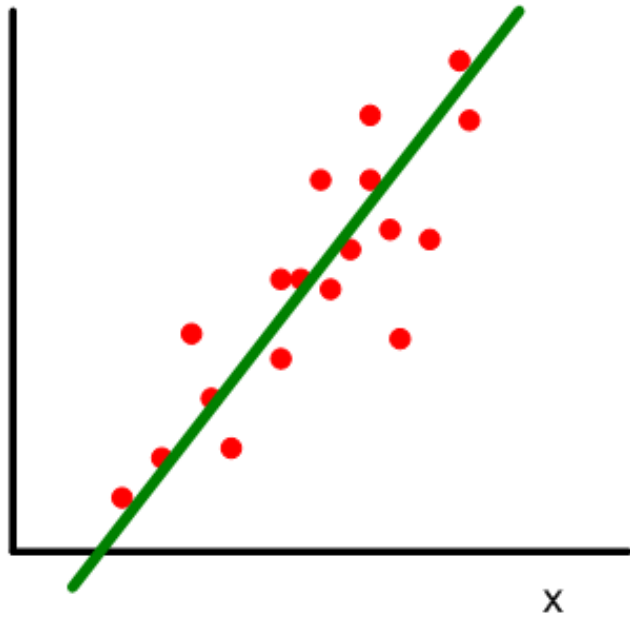
Regression is an area of statistical modeling.

It estimates the relationships between a **dependent variable** (outcome, response variable) and one or more **independent variables** (predictors, covariates, explanatory variables, features).



Introduction

Linear Regression is a **supervised** machine learning algorithm where the predicted output is continuous and has a constant slope. It's used to predict values within a continuous range, (e.g. price) rather than trying to classify them into categories (e.g. cat, dog). There are two main types:



- **univariate** regression: $y = k\,x + q$
- **multivariate** regression, e.g. $y = w_1x_1 + w_2x_2 + w_3x_3 + b$

The regression is a part of data analysis, where most univariate analysis emphasizes description of our data, while multivariate methods emphasize hypothesis testing and explanation (including feature analysis).

Linear regression

Linear regression

Data regression problems comes in the form of a (training) dataset of N observation pairs:

$$\{[\mathbf{x'}_1, y_1], [\mathbf{x'}_2, y_2], \dots [\mathbf{x'}_n, y_n]\}$$

where $\mathbf{x'}_i$ denotes the input variables and y_i denotes the output. Both variables are generally continuous.

The simplest case is, if input variable is a scalar - in that case we are solving 2-dimensional problem, i.e. fitting a line in 2D space through scattered data. In general, however, each input $\mathbf{x'}_i$ may be a collum vector of length d:

$$\mathbf{x}'_i = \left[x_{i,1}\ x_{i,2} \dots x_{i,d}\right]^T$$

d is a dimension of input vector, number of features

n is a number of datapoints

And all samples can be organized to feature matrix and label vector:

$$\mathbf{X}' = \begin{bmatrix} x_{11} & \cdots & x_{1d} \\ \vdots & \ddots & \vdots \\ x_{n1} & \cdots & x_{nd} \end{bmatrix} \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$$

Linear regression

In this case the linear regression problem is fitting a hyperplane to a scatter of points in d+1 dimensional space.

In the case of scalar input, fitting a line to the data requires that we determine a slope *w* and a bias *b* of the regression line so that the approximate linear relationship holds between input/output data:

$$y_i \approx x_i w + b, \qquad i = 1, 2, \ldots, n$$

Linear regression

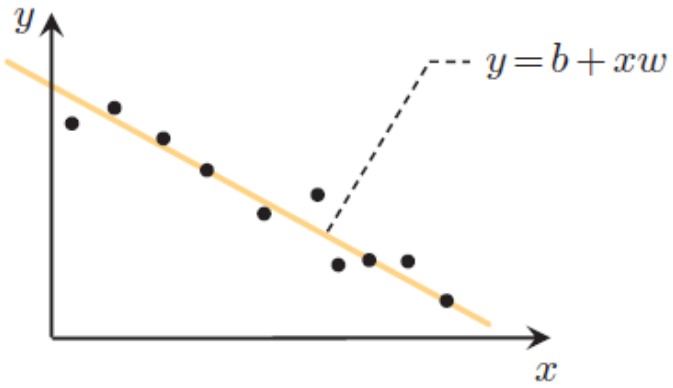
More generally, when the input dimension is d ≥ 1, we have a bias and d associated weights:

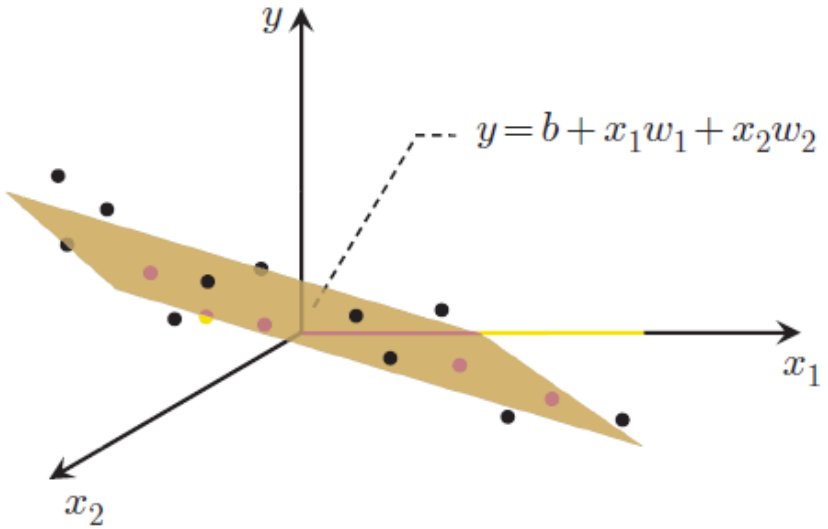
The weights determine the normal vector of the hyperplane:

$$\mathbf{w}' = [w_1\ w_2\ \ldots w_d]^T$$

The sample can be written as a vector:

$$\mathbf{x}_i' = [x_1\ x_2\ \ldots x_d]^T$$





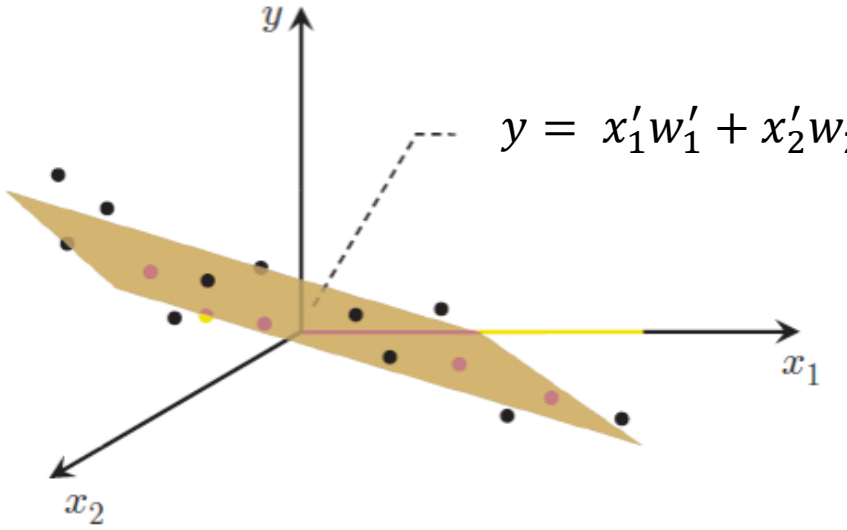
Linear regression

$$\mathbf{w} = \begin{bmatrix} b \\ \mathbf{w}' \end{bmatrix}$$

For simplification of equations, we can define:

$$\mathbf{x_i} = \begin{bmatrix} 1 \\ \mathbf{x}'_\mathbf{i} \end{bmatrix}$$

These “new” vectors are called **augmented** vectors.



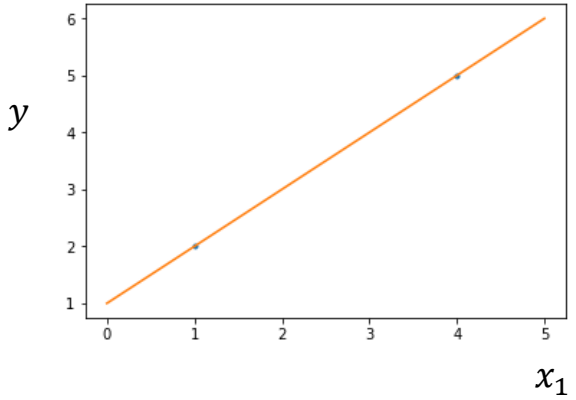
$$y = x'_1 w'_1 + x'_2 w'_2 = 1w_0 + x_1 w_1 + x_2 w_2 = \mathbf{w^T x}$$

And “new” feature matrix:

$$\mathbf{X} = \begin{bmatrix} 1 & x_{11} & \cdots & x_{1d} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & \cdots & x_{nd} \end{bmatrix}$$

It allows to use linear equations to fit affine functions.

Linear regression



Let assume simple case with just two 2D points:

$$\mathbf{x_1} = \begin{bmatrix} 1 \\ x_{11} \end{bmatrix} \quad \mathbf{x_2} = \begin{bmatrix} 1 \\ x_{21} \end{bmatrix}$$

$$\mathbf{X} = \begin{bmatrix} 1 & x_{11} \\ 1 & x_{21} \end{bmatrix} \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \end{bmatrix}$$

Points need to satisfy line equation „*y = kx + q*“, thus we need to find weights which satisfy:

$$y_1 = x_{11}w_1 + w_0$$
$$y_2 = x_{21}w_1 + w_0$$

…two equations and two unknowns…we can solve for weights.

Matrix form:

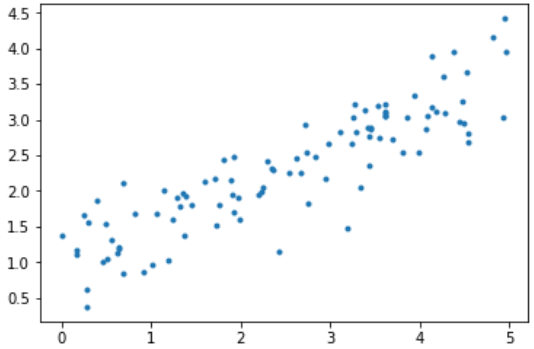
w than can be found as solution of $\mathbf{y} = \mathbf{Xw}$

$$\mathbf{w} = \mathbf{X^{-1}y}$$

For prediction, we can use $\mathbf{y_{new}} = \mathbf{X_{new}w}$ where $\mathbf{X_{new}}$ is feature matrix of new samples.

($y_{new} = \mathbf{w^T x_{new}}$ for one sample)

Linear regression



But what in case of more samples, where $\mathbf{y} = \mathbf{Xw}$
can‘t be satisfied for all points (line can‘t go through all points)?

$\mathbf{y}$ can‘t be equal to $\mathbf{Xw}$, but we can make it similar:

$$\underset{\boldsymbol{w}}{minimize}\ g(\mathbf{w}) = \sum_{i=1}^{p} (\mathbf{x_i^T w} - y_i)^2 = ||\mathbf{Xw} - \mathbf{y}||^2$$

We can use some iterative solver (e.g. gradient descent), but there is simple closed form solution.

The derivative of this cost function with respect to **w** is :

$$\nabla g(\mathbf{w}) = \sum_{i=1}^{p} 2(\mathbf{w}^T \mathbf{x_i} - y_i)\mathbf{x_i} = 2\mathbf{X^T}(\mathbf{Xw} - \mathbf{y})$$

Here, we can set the gradient to zero:

$$2\mathbf{X^T}(\mathbf{Xw} - \mathbf{y}) = 0$$

We are looking for **w**:

$$\mathbf{X^T X w} = \mathbf{X^T y}$$

And finally:

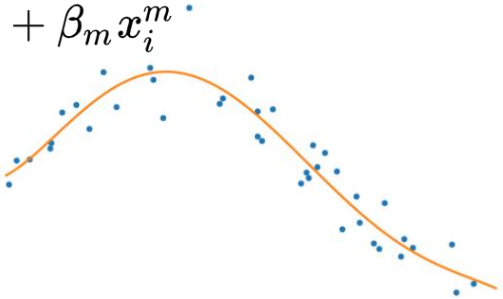
$$\mathbf{w} = (\mathbf{X^T X})^{-1}\mathbf{X^T y} = \mathbf{X}^{\dagger}\mathbf{y}$$

This is a direct solution, which needs the inverse matrix of $\mathbf{X^T X}$, which is square and often nonsingular (one that has inverse matrix). Term $\mathbf{(X^T X)^{-1} X^T}$ is also known as pseudo-inverse.

Polynomial regression

We can use same approach as for linear regression to fit different (non-linear) function.

e.g. m-th degree polynomial: $y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \cdots + \beta_m x_i^m$



Which is equivalent to transforming features and then using linear regression:

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} 1 & x_1 & x_1^2 & \dots & x_1^m \\ 1 & x_2 & x_2^2 & \dots & x_2^m \\ 1 & x_3 & x_3^2 & \dots & x_3^m \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \dots & x_n^m \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \vdots \\ \beta_m \end{bmatrix},$$

Estimation of regression coefficient is formally the same as for linear regression.
Only the **X** matrix is computed beforehand.

Feature transformation

We can define feature transformation to higher dimensional space D: $\Phi: R^d \rightarrow R^D$

And replace samples in feature matrix with transformed features: $\boldsymbol{X} = \begin{bmatrix} \Phi(\mathbf{x_1})^T \\ ... \\ \Phi(\mathbf{x_n})^T \end{bmatrix}$

For example, second order polynomial feature transformation of 2D vector is

$$\Phi([x_1, x_2]^T) = [x_1^2, x_2^2, x_1 x_2, x_1, x_2, 1]^T$$

With those transformations, we can easily fit polynomials and some other nonlinear functions.

This is general way how to make (linear) model nonlinear and more powerful – able to fit more complex relations, but more prone to overfitting (can be overcame with regularization).

It also largely increases componential complexity, which can be overcame with kernel trick.

Norms

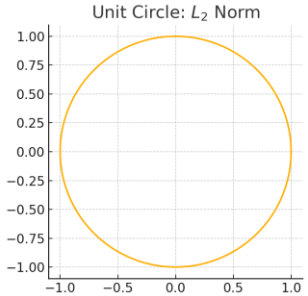
Lp-norm $\|\mathbf{x}\|_p = \left(\sum_{i=1}^{n} |x_i|^p\right)^{1/p}$

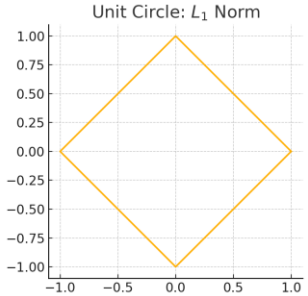
L2-norm $\|\mathbf{x}\| = \|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^{n} x_i^2}$ „Euklidean distance“

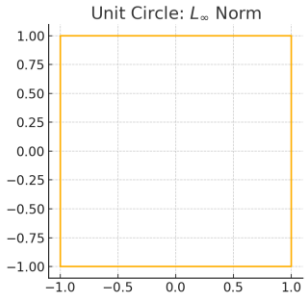
$\|\mathbf{x}\|_2^2 = \mathbf{x}^\mathbf{T}\mathbf{x}$

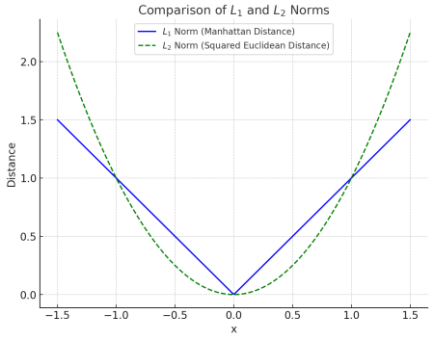
L1-norm $\|\mathbf{x}\|_1 = \sum_{i=1}^{n} |x_i|$ „ manhattan distance“

L∞ -norm $\|\mathbf{x}\|_\infty = \max_i |x_i|$ „ Chebyshev distance“









Regularization

Regularization - general

Regularization is a technique used in the optimization to prevent overfitting and improve the generalization of a model. It adds a *penalty term* to the loss function during training.

- Prevent overfitting
- Improve generalization
- Handling Multicollinearity - in regression problems with a large number of features, multicollinearity (high correlation between predictor variables) can lead to unstable coefficient estimates. Ridge regression (L2 regularization) can handle multicollinearity by shrinking the coefficients of correlated features.
- Feature Selection and model complexity – see Lasso regression later.
- Stability and Numerical Robustness - important for iterative optimization algorithms, ensuring that they converge more reliably.

(Linear) Ridge regression

In a case of high number of input variables we may wish to decrease the model complexity, that is the number of predictors (input variables). We could use some methods for feature selection - e.g. forward or backward selection. But we can do it also during optimization by introducing a new term.

Removing predictors from the model can be seen as settings their coefficients to zero. Instead of forcing them to be exactly zero, we can penalize them if they are too far from zero, thus enforcing them to be small in a continuous way. This way, we decrease model complexity while keeping all variables in the model. This, basically, is what ridge regression does.

(Linear) Ridge regression

This extra regularization term penalizes the sum of the squared coefficients, which discourages coefficients from taking on extremely large values.

The regularized version is:

$$g_{ridge}(\mathbf{w}) = \sum_{i=1}^{p}(\mathbf{w}^T\mathbf{x_i} - y_i)^2 + \lambda \sum_{i=1}^{n} w_i^2 = ||\mathbf{X}\mathbf{w} - \mathbf{y}||^2 + \lambda||\mathbf{w}||^2$$

where $||\mathbf{w}||^2 = \sum w_i^2$ is the squared norm of **w** or equivalently, the dot product $\mathbf{w^Tw}$, λ is scalar value determining the level of regularization. This L^2 norm regularization is often called as **ridge** regularization, thus ridge regression.

The λ parameter is the regularization penalty:

$$\lambda \to 0, \quad \mathbf{w}_{ridge} \to \mathbf{w}$$

$$\lambda \to \infty, \quad \mathbf{w}_{ridge} \to 0$$

(Linear) Ridge regression

This form of regularization still have a closed form solution:

$$\mathbf{w} = (\mathbf{X^T X} + \lambda \mathbf{I})^{-1} \mathbf{X^T y}$$

where $\mathbf{I}$ is identity matrix. If we compare this equation with equation for **w** without regularization, we see that here we add λ to the diagonal of $\mathbf{X^T X}$, which is know trick to improve the stability of matrix inversion. This, so called, ill-conditioned matrix can occur for correlates samples.

The problem with inversion can also occur if we have small number of samples (with respect to number of features).

(Linear) Lasso Regression

Lasso (Least Absolute Shrinkage and Selection Operator), is quite similar conceptually to ridge regression. It also adds a penalty for non-zero coefficients, but unlike ridge regression which penalizes sum of squared coefficients (L2 penalty), lasso penalizes the sum of their absolute values (L1 penalty). As a result, for high values of λ, many coefficients are exactly zeroed under lasso, which is never the case in ridge regression.
The cost function has this form:

$$g_{lasso}(\mathbf{w}) = \sum_{i=1}^{n} (\mathbf{w}^T \mathbf{x} - y)^2 + \lambda \sum_{i=1}^{p} |w_i| = ||\mathbf{X}\mathbf{w} - \mathbf{y}||^2 + \lambda ||\mathbf{w}||_1$$

where $||\mathbf{w}||_1 = \sum_{i=1}^{n} |w_i|$

No closed form solution. Numerical optimization.

More information about ridge and lasso regularization: https://www.youtube.com/watch?v=sO4ZirJh9ds

Least Absolute Deviation (LAD) Regression

Least squares are sensitive to outliers -> sometimes it is better to minimize MAE loss (instead of MSE loss). It is LAD/L1 regression.

$$g_{LAD}(w) = \|Xw - y\|_1 (+regularzation)$$

No closed form solution, but can be solved iteratively with e.g. gradient descent.

Less sensitive to outliers.

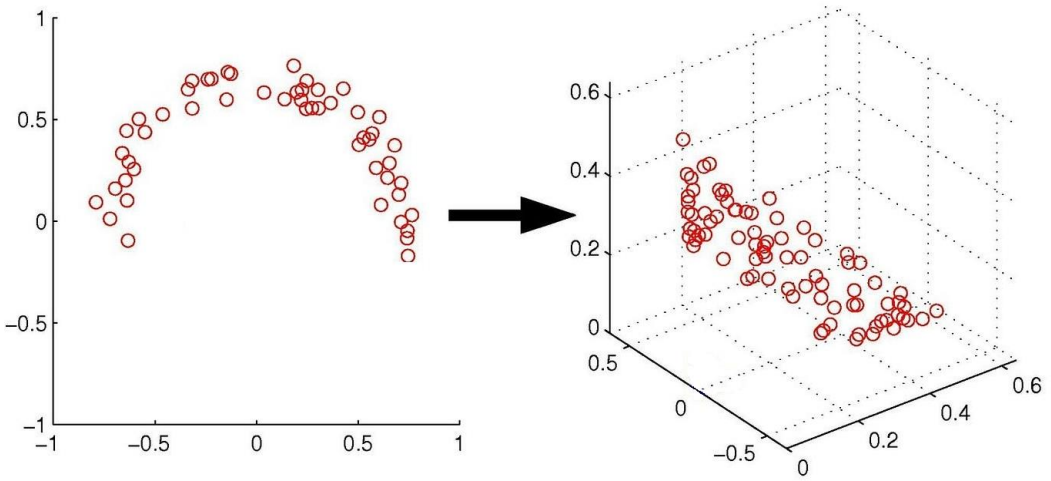
Kernels

Feature tranformation

In the example, the original data is in 2-dimension. Suppose we denote it as, $\mathbf{x}=[x_1, x_2]$. We can see *non-linear* distribution of the samples. But they can be transformed to space with higher dimensionality and they form linear:

$$\Phi(\boldsymbol{x}) = x_1^2, x_2^2, \sqrt{2}x_1x_2$$

where, $\boldsymbol{\Phi}$ is a transform function from 2-D to 3-D applied on $\mathbf{x}$.



In general, we map original data from *m* dimensional space to *p* dimensional space, m<p. And we hope in *better* representation of the data in this new space.

In this new space we can perform the regression, but with *m* number of features. However, this is not possible and/or efficient for too high m (thousands).

Kernel trick

Fortunately, there is a kernel trick! In many machine learning methods, the **dot product** over the data is applied. The results of the dot product of two vectors is a scalar value. So, consider the dot product of two vectors (in 2 dimensions):

$$\mathbf{x_i^T x_j} = \langle \mathbf{x_i}, \mathbf{x_j} \rangle = \langle \begin{bmatrix} x_{i1} \\ x_{i2} \end{bmatrix}, \begin{bmatrix} x_{j1} \\ x_{j2} \end{bmatrix} \rangle = x_{i1}x_{j1} + x_{i2}x_{j2}$$

Now, consider the dot product after transforming the features into higher dimension (from previous example):

$$\langle \phi(\mathbf{x_i}), \phi(\mathbf{x_j}) \rangle = \langle \begin{bmatrix} x_{i1}^2 \\ x_{i2}^2 \\ \sqrt{2}x_{i1}.x_{i2} \end{bmatrix}, \begin{bmatrix} x_{j1}^2 \\ x_{j2}^2 \\ \sqrt{2}x_{j1}.x_{j2} \end{bmatrix} \rangle = x_{i1}^2 x_{j1}^2 + x_{i2}^2 x_{j2}^2 + 2x_{i1}x_{i2}x_{j1}x_{j2}$$

Kernel trick

The disadvantage of this approach is that we first need to transform all the feature set into high dimension. And because we typically have many features (not only two) it creates very high dimensional space (p>>m) and therefore it is computationally demanding and also the memory storage increases.

Fortunately, we can do it in a smarter way:

$$\langle \mathbf{x_i}, \mathbf{x_j} \rangle^2 = \langle \begin{bmatrix} x_{i1} \\ x_{i2} \end{bmatrix}, \begin{bmatrix} x_{j1} \\ x_{j2} \end{bmatrix} \rangle^2 =$$

$$(x_{i1}x_{j1} + x_{i2}x_{j2})^2 = x_{i1}^2 x_{j1}^2 + x_{i2}^2 x_{j2}^2 + 2x_{i1}x_{i2}x_{j1}x_{j2}$$

So, we are able to compute the dot product **without** explicitly going to higher dimension!

So what is the kernel?

Suppose we have a mapping Φ: $R_n \rightarrow R_m$ that brings our vectors in R_n to some feature space R_m. Then the dot product of **x** and **y** in this space is $\Phi(\mathbf{x})^T \Phi(\mathbf{y})$. A kernel is a function *k* that corresponds to this dot product:

$$k(\mathbf{x},\mathbf{y}) = \Phi(\mathbf{x})^T \Phi(\mathbf{y})$$

Kernel	Kernel function, $k(x, y)$
Linear	$x^\top y$
Poly	$(\gamma x^\top y + c_0)^p$
RBF	$\exp(-\gamma \|x - y\|^2)$
Tanh	$\tanh(\gamma\, x^\top y + c_0)$
ARD	$v^2 \exp\left(-\frac{1}{2}\sum_{d=1}^{D}\left(\frac{x^d - y^d}{\lambda_d}\right)^2\right)$

https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0235885

And we can define kernel matrix:

$$\mathbf{K} = \phi(\mathbf{X})\phi(\mathbf{X})^T = \begin{bmatrix} \kappa(\mathbf{x_1},\mathbf{x_1}) & \kappa(\mathbf{x_1},\mathbf{x_2}) & \dots & \kappa(\mathbf{x_1},\mathbf{x_p}) \\ \kappa(\mathbf{x_2},\mathbf{x_1}) & \kappa(\mathbf{x_2},\mathbf{x_2}) & \dots & \kappa(\mathbf{x_2},\mathbf{x_p}) \\ \vdots & \vdots & \ddots & \vdots \\ \kappa(\mathbf{x_p},\mathbf{x_1}) & \kappa(\mathbf{x_p},\mathbf{x_2}) & \dots & \kappa(\mathbf{x_p},\mathbf{x_p}) \end{bmatrix}$$

Polynomial kernel

A polynomial mapping is a popular method for non-linear modelling. The features are transformed to Q-dimensional space and the dot product is computed. This can be compute directly via kernel function:

$$k\left(\mathbf{x_i}, \mathbf{x_j}\right) = \left(1 + \mathbf{x_i^T x_j}\right)^Q$$

The Q value can be 1, 2, … up to some finite number. This is set by user when specifying the kernel type. Kernel matrix can be calculated using:

$$\mathbf{K} = \left(\mathbf{X X^T} + 1\right)^{\mathrm{Q}}$$

(Gaussian) Radial Basis Function

We can also go to infinite dimensions… so we can compute the dot product for infinite dimension via kernel trick. How is it possible? Consider this Gaussian function:

$$k\left(\mathbf{x_i}, \mathbf{x_j}\right) = e^{-\frac{\left(\mathbf{x_i}-\mathbf{x_j}\right)^2}{2\sigma^2}}$$

Consider the simple 1D feature vector and ignore σ we can rewrite this function using Taylor series:

$$k\left(\mathbf{x_i}, \mathbf{x_j}\right) = e^{-\frac{\left(\mathbf{x_i}-\mathbf{x_j}\right)^2}{2\sigma^2}} = e^{-x_i^2} e^{-x_j^2} \sum_{k=0}^{\infty} \frac{2^k x_i^k x_j^k}{k!} =$$

$$= e^{-x_i^2} e^{-x_j^2} . \left(1 + 2x_i x_j + 2x_i^2 x_j^2 + \frac{8}{6} x_i^3 x_j^3 + \ldots\right)$$

(Gaussian) Radial Basis Function

We can also go to infinite dimensions… so we can compute the dot product for infinite dimension via kernel trick. How is it possible? Consider this Gaussian function:

$$k\left(\mathbf{x_i}, \mathbf{x_j}\right) = e^{-\frac{(\mathbf{x_i}-\mathbf{x_j})^2}{2\sigma^2}}$$

Consider the simple 1D feature vector and ignore σ we can rewrite this function using Taylor series :

$$k\left(\mathbf{x_i}, \mathbf{x_j}\right) = e^{-\frac{(\mathbf{x_i}-\mathbf{x_j})^2}{2\sigma^2}} = e^{-x_i^2} e^{-x_j^2} \sum_{k=0}^{\infty} \frac{2^k x_i^k x_j^k}{k!} =$$

$$= e^{-x_i^2} e^{-x_j^2} \cdot \left(1 + 2x_i x_j + 2x_i^2 x_j^2 + \frac{8}{6} x_i^3 x_j^3 + \ldots\right)$$

=> corresponding transformed features space is ∞-dimensional.

How to kernelize some linear method?

- Modify method to dual form, where all features will be in form of dot product $\mathbf{x_i^T x_j}$ or Gramm matrix $\mathbf{XX^T}$ (containing dot products).
  - original weights w are replaced with sample weights α
- Replace dot product and Gram matrices with kernel function k($\mathbf{x_i^T x_j}$) and kernel matrix K

Kernel ridge regression

Kernel ridge regression

$$\mathbf{X} = \begin{bmatrix} 1 & x_{11} & \cdots & x_{1n} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{p1} & \cdots & x_{pn} \end{bmatrix}$$

The kernel approach can be used also here to make the regression non-linear. And, of course, kernel trick can be used.

Let's start with ridge regression. The solution still have closed form:

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ . \\ . \\ y_p \end{bmatrix}$$

$$\mathbf{w} = (\mathbf{X^T X} + \lambda \mathbf{I})^{-1} \mathbf{X^T y}$$

There is some mathematical (matrix inversion or Woodbury) lemma, which allows us to rewrite this solution to dual form:

$$\alpha = \begin{bmatrix} \alpha_1 \\ \alpha_2 \\ . \\ . \\ \alpha_p \end{bmatrix}$$

$$\mathbf{w} = \mathbf{X^T}(\mathbf{X X^T} + \lambda \mathbf{I})^{-1} \mathbf{y} = \mathbf{X^T} \alpha = \sum_{i=1}^{p} \alpha_i \mathbf{x_i}$$

where

$$\alpha = (\mathbf{X X^T} + \lambda \mathbf{I})^{-1} \mathbf{y} = (K + \lambda \mathbf{I})^{-1} \mathbf{y}$$

is an p x 1 vector of **dual variables**, and $\mathbf{K_{ij} = x_i^T x_j}$.

p - number of samples

Kernel ridge regression

Note that $$\mathbf{w} = \sum_{i=1}^{p} \alpha_i \mathbf{x_i} = \mathbf{X^T} \alpha$$

It is known as **dual form** of ridge regression solution. However, it is still a linear model. But, it can be easily *kernelized and thus we can use kernel trick*.

In primal form, we need to invert *dxd* matrix, where d is number of features. We have *d+1* weights w for individual features*:*

$$\mathbf{w} = (\mathbf{X^T X} + \lambda \mathbf{I})^{-1} \mathbf{X^T y}$$

In dual form, we need to invert *nxn,* where *n* is number of samples. We have *n* weights α for individual samples:

$$\alpha = (\mathbf{X X^T} + \lambda \mathbf{I})^{-1} \mathbf{y} = (K + \lambda \mathbf{I})^{-1} \mathbf{y}$$

Kernel ridge regression

Let’s go to high dimensional space. Choosing some kernel function *k* with an associated feature mapping Φ, we can write

$$\mathbf{w} = \sum_{i=1}^{p} \alpha_i \phi(\mathbf{x_i}) = \phi(\mathbf{X})^T \alpha$$

Prediction for new test input **x** will be:

$$\mathbf{y} = \phi(\mathbf{x})^T \mathbf{w} = \mathbf{w^T} \phi(\mathbf{x}) = \sum_{i=1}^{p} (\alpha_i \phi(\mathbf{x_i}))^T \phi(\mathbf{x}) = \sum_{i=1}^{p} \alpha_i k(\mathbf{x_i}, \mathbf{x})$$

Kernel ridge regression

Thus for our new input sample, we can effectively compute the output using some kernel function (which we choose before any computation) and we can compute the dot products between each sample from our training set and new input.

But, how to compute α_i ?

Kernel ridge regression

Let's write the cost function for non-linear case:

$$g_{kernel}(\mathbf{w}) = ||\phi(\mathbf{X})\mathbf{w} - \mathbf{y}||^2 + \lambda||\mathbf{w}||^2$$

And substitute $\mathbf{w} = \phi(\mathbf{X})^T\alpha$

Dot products in high dimension

$$g_{kernel}(\alpha) = ||\phi(\mathbf{X})\phi(\mathbf{X})^{\mathbf{T}}\alpha - \mathbf{y}||^2 + \lambda\alpha^{\mathbf{T}}\phi(\mathbf{X})\phi(\mathbf{X})^{\mathbf{T}}\alpha$$

Let's write K instead of $\Phi(x)\ \Phi(x)^T$:

$$g_{kernel}(\alpha) = ||\mathbf{K}\alpha - \mathbf{y}||^2 + \lambda\alpha^{\mathbf{T}}\mathbf{K}\alpha =$$
$$\alpha^{\mathbf{T}}\mathbf{K}\mathbf{K}^{\mathbf{T}}\alpha - 2\mathbf{y}^{\mathbf{T}}\mathbf{K}\alpha + \mathbf{y}^{\mathbf{T}}\mathbf{y} + \lambda\alpha^{\mathbf{T}}\mathbf{K}\alpha$$

Kernel ridge regression

Taking the gradient with respect to α and setting it to zero:

$$\nabla_\alpha g_{kernel}(\alpha) = 2\mathbf{K}\mathbf{K}^{\mathbf{T}}\alpha - 2\mathbf{K}\mathbf{y} + 0 + 2\lambda\mathbf{K}\alpha$$

leads to final result:

$$\boxed{\alpha = (\mathbf{K} + \lambda\mathbf{I})^{-1}\mathbf{y}}$$

$$\mathbf{K} = \phi(\mathbf{X})\phi(\mathbf{X})^T = \begin{bmatrix} \kappa(\mathbf{x_1}, \mathbf{x_1}) & \kappa(\mathbf{x_1}, \mathbf{x_2}) & \dots & \kappa(\mathbf{x_1}, \mathbf{x_p}) \\ \kappa(\mathbf{x_2}, \mathbf{x_1}) & \kappa(\mathbf{x_2}, \mathbf{x_2}) & \dots & \kappa(\mathbf{x_2}, \mathbf{x_p}) \\ \vdots & \vdots & \ddots & \vdots \\ \kappa(\mathbf{x_p}, \mathbf{x_1}) & \kappa(\mathbf{x_p}, \mathbf{x_2}) & \dots & \kappa(\mathbf{x_p}, \mathbf{x_p}) \end{bmatrix}$$

Kernel PCA

Covariance matrix for PCA: $\mathbf{C} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{x}_i \mathbf{x}_i^\top .$

Kernel PCA

Now, let's apply this *kernel trick* to PCA. We must rewrite it in order to obtain the dot product in the eigendecomposition.

We start with covariance matrix of the projected features (size p x p), calculated as

$$\mathbf{C} = \frac{1}{n}\sum_{i=1}^{n} \phi(\mathbf{x_i})\phi(\mathbf{x_i})^T$$

Its eigenvalues and eigenvectors are given by:

$$\mathbf{C}q_k = \lambda_k q_k,$$

for k = 1,...p.

p is the number of features after transforming the features

We would like to do kernel PCA without having to compute C or ***Φ(x)***. We will use kernel trick.

Kernel PCA

k - index for number of principal component.

Substitute the definition of C into last equation:

$$\frac{1}{n}\sum_{i=1}^{n}\phi(\mathbf{x_i})[\phi(\mathbf{x_i})^T\mathbf{q_k}] = \lambda_k\mathbf{q_k}.$$

The term in bracket gives a single number (multiplication of row and column vector) and dividing this number by $\boldsymbol{\lambda}_k$ and *n* gives new variable a_{ki}. It can be then written as:

$$\mathbf{q_k} = \sum_{i=1}^{n} a_{ki}\phi(\mathbf{x_i}).$$

The eigenvectors are linear combination of transformed features.

Now, substituting this equation into previous equation leads to:

$$\frac{1}{n}\sum_{i=1}^{n}\phi(\mathbf{x_i})\phi(\mathbf{x_i})^T\sum_{j=1}^{n}a_{kj}\phi(\mathbf{x_j}) = \lambda_k\sum_{i=1}^{n}a_{ki}\phi(\mathbf{x_i})$$

and multiply both sides of the last equation by $\boldsymbol{\phi}(\mathbf{x}_l)^T$, we have:

$$\frac{1}{n}\sum_{i=1}^{n}\phi(\mathbf{x_l})^T\phi(\mathbf{x_i})\phi(\mathbf{x_i})^T\sum_{j=1}^{n}a_{kj}\phi(\mathbf{x_j}) = \lambda_k\sum_{i=1}^{n}a_{ki}\phi(\mathbf{x_l})^T\phi(\mathbf{x_i}).$$

Kernel PCA

This is dot product between transformed samples - can be computed via kernel trick.

If we define the kernel function: $\kappa(\mathbf{x_i}, \mathbf{x_j}) = \phi(\mathbf{x_i})^T \phi(\mathbf{x_j}),$

we can rewrite the last equation to

$$\frac{1}{n} \sum_{i=1}^{n} \kappa(\mathbf{x_l}, \mathbf{x_i}) \sum_{j=1}^{n} a_{kj} \kappa(\mathbf{x_i}, \mathbf{x_j}) = \lambda_k \sum_{i=1}^{n} a_{ki} \kappa(\mathbf{x_l}, \mathbf{x_i}).$$

Number of datapoints. Size of matrix K is n.

Define K as a *n x n* kernel matrix:

$$K = \phi(\mathbf{X})\phi(\mathbf{X})^T = \begin{bmatrix} \kappa(\mathbf{x_1}, \mathbf{x_1}) & \kappa(\mathbf{x_1}, \mathbf{x_2}) & \dots & \kappa(\mathbf{x_1}, \mathbf{x_n}) \\ \kappa(\mathbf{x_2}, \mathbf{x_1}) & \kappa(\mathbf{x_2}, \mathbf{x_2}) & \dots & \kappa(\mathbf{x_2}, \mathbf{x_n}) \\ \vdots & \vdots & \ddots & \vdots \\ \kappa(\mathbf{x_n}, \mathbf{x_1}) & \kappa(\mathbf{x_n}, \mathbf{x_2}) & \dots & \kappa(\mathbf{x_n}, \mathbf{x_n}) \end{bmatrix}$$

The n-elements in each column of K represent the dot product of one sample of the transformed data with respect to all the transformed samples (n samples).

Kernel function

Define $\mathbf{a_k}$ is the n-dimensional column vector: $a_k = \begin{bmatrix} a_{k1} \\ a_{k2} \\ \vdots \\ a_{kn}. \end{bmatrix}$

With this matrix notation we can write:

$$\mathbf{K}^2\mathbf{a_k} = \lambda_k n \mathbf{K}\mathbf{a_k}$$

and finally $\mathbf{a_k}$ and $\boldsymbol{\lambda}_k$ can be solved by:

$$\mathbf{K}\mathbf{a_k} = \lambda_k n \mathbf{a_k},$$

λ_k are the eigenvalues, $\boldsymbol{a_k}$ are eigenvectors, both connected with covariance matrix **K**.

which is similar to linear PCA, for which we had:

$$\mathbf{C}\,\mathbf{q}_j = \lambda_j\,\mathbf{q}_j, \quad j = 1, \ldots, m.$$

Kernel PCA

In linear PCA, we the data must be centered before computing the covariance matrix.

For kernel PCA, we need to do the same, because the after application of kernel, the matrix is generally not centered.

In the matrix notation, the centered K is:

$$\tilde{\mathbf{K}} = \mathbf{K} - \mathbf{1_n}\mathbf{K} - \mathbf{K}\mathbf{1_n} + \mathbf{1_n}\mathbf{K}\mathbf{1_n}$$

Where $\mathbf{1_n}$ is n x n matrix with every element = 1/n.

Eigen-decomposition is then done for the centered kernel matrix.

Summary of the algorithm

1. Construct the n × n kernel matrix **K**.
2. Center **K.**
3. Do eigendecomposition of centered **K** and find top *L* eigenvectors $\mathbf{a_1}$,...$\mathbf{a_L}$ with eigenvalues $\lambda_1, \ldots, \lambda_L$.
4. For any data point (new or old) **x**, we can represent it as the following set of features:

$$y_k = \sum_{i=1}^{n} a_{ki} \kappa(\mathbf{x}, \mathbf{x_i})$$

for k = 1,...L.

PCA, notes

- Kernel PCA can give a good re-mapping of the data when it lies along a non-linear “space”.
- The kernel matrix is n × n, so kernel PCA will have difficulties if we have lots of data points.
- Results depends on the kernel.

PCA

Primal form

$$\mathbf{C} = \frac{1}{N}\mathbf{X}^\top\mathbf{X} = \frac{1}{N}\sum_{i=1}^{N}\mathbf{x_i}\mathbf{x_i}^T$$

Eigenvalue decomposition:

$$\lambda\mathbf{w} = \frac{1}{N}\sum_{i=1}^{N}\mathbf{x_i}\mathbf{x_i}^T\mathbf{w}$$

Projection of new datapoint $\mathbf{x_{new}}$ it k-th PC:

$$t_k = \mathbf{x_{new}}^T\mathbf{w_k}$$

$$\mathbf{T} = \mathbf{XW}$$

Dual form

$$\mathbf{C'} = \mathbf{X}\mathbf{X}^\top = \sum_{i=1}^{N}\mathbf{x_i}^T\mathbf{x_i}$$

Eigenvalue decomposition:

$$N\mathbf{\Lambda a} = \sum_{i=1}^{N}\mathbf{x_i}^T\mathbf{x_i a}$$

Projection of new datapoint $\mathbf{x_{new}}$ to k-th PC:

$$t_k = \frac{1}{\sqrt{\lambda}_k}\sum_{i=1}^{N}a_{ik}\mathbf{x_{new}}^T\mathbf{x_i} = \frac{1}{\sqrt{\lambda}_k}\mathbf{a_k^T X x_{new}^T}$$

Linear regression

Primal form

$$\underset{\mathbf{w}}{\text{mininize}} ||\mathbf{X}\mathbf{w} - \mathbf{y}||^2 + \lambda ||\mathbf{w}||^2$$

closed form solution:

$$w^* = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^T\mathbf{y}$$

$$\hat{y} = \mathbf{X}\mathbf{w}$$

Dual form

$$\underset{\alpha}{\text{mininize}} \quad \tfrac{1}{2}\alpha^\top(\mathbf{X}\mathbf{X}^T + \lambda\mathbf{I})\alpha - \mathbf{y}^\top\alpha$$

$$\alpha = (\mathbf{X}\mathbf{X}^T + \lambda\mathbf{I})^{-1}\mathbf{y}$$

$$\mathbf{w} = \sum_{i=1}^{N} \alpha_i \mathbf{x_i}$$

then prediction of new sampe

$$\hat{y} = \sum_{i=1}^{N} \alpha_i \mathbf{x_{new}}\mathbf{x_i}^T = \alpha^\mathbf{T}\mathbf{X}\mathbf{x_{new}^T}$$

SVM

Primal form

training:

$$\underset{\mathbf{w}}{\text{mininize}}\ \frac{1}{2}\mathbf{w}^T\mathbf{w} + c\sum_j \xi_j$$

$$\text{subject to } (\mathbf{w}^T\mathbf{x_j} + b)y_j \geq 1 - \xi_j$$

prediction:

$$\hat{y} = \text{sign}(\mathbf{w}^T\mathbf{x_{new}} + b)$$

Dual form

training:

$$\underset{\alpha}{\text{maximize}}\ \sum_{i=1}^{N}\alpha_i \sum_{i=1}^{N}\sum_{j=1}^{N}\alpha_i$$

$$-\frac{1}{2}\alpha_j y_i y_i \mathbf{x_i}^T\mathbf{x_j}$$

$$\text{subject to } 0 \leq \alpha_i \leq c$$

$$\sum_{i=1}^{N}\alpha_i y_i = 0$$

prediction:

$$\hat{y} = \text{sign}(\sum_{k\in SV}\alpha_k y_k \mathbf{x}_{new}^T\mathbf{x}_k + b)$$